Лабораторная работа №5 по курсу дискретного анализа: поиск с использованием суффиксного массива

Выполнил студент группы 08-215 МАИ *Матвеев Семен*.

Условие

Необходимо найти в заранее известном тексте поступающие на вход образцы с использованием суффиксного массива.

Формат входа:

- первая строка содержит текст;
- затем, до конца файла, следуют строки с образцами.

Для каждого образца, найденного в тексте, нужно распечатать строку, начинающуюся с последовательного номера этого образца и двоеточия. После двоеточия через запятую перечисляются номера позиций, где образец встречается в тексте, в порядке возрастания. Нумерация позиций начинается с единицы. Если образец не встречается в тексте, строка для него не выводится.

Вариант задания:

- алгоритм: поиск с использованием суффиксного массива;
- текст: первая строка входного файла;
- образцы: все последующие строки до конца файла.

Метод решения

Для решения задачи используется суффиксный массив. Суффиксный массив — это массив позиций всех суффиксов строки, отсортированных в лексикографическом порядке. Если построить такой массив для исходного текста, то поиск образца можно свести к двум бинарным поискам по массиву суффиксов.

Основная идея

1. Сначала считывается первая строка входного файла — это основной текст, в котором дальше будут искаться образцы.
2. Для текста строится суффиксный массив. В реализации используется алгоритм сортировки циклических сдвигов с удвоением длины рассматриваемой подстроки.
3. К тексту временно добавляется символ `'\0'`, который меньше всех обычных символов. Это позволяет корректно построить суффиксный массив через сортировку циклических сдвигов.

4. На первом шаге суффиксы сортируются по первому символу. Затем на каждом следующем шаге сортируются пары классов эквивалентности для подстрок текущей длины `substringLength`, которая каждый раз удваивается.

5. После построения из массива удаляется позиция фиктивного нулевого символа, потому что она не является суффиксом исходного текста.

6. Для каждого образца выполняются два бинарных поиска:

   - `LowerBound` находит первый суффикс, который не меньше образца;
   - `UpperBound` находит первый суффикс, который строго больше образца с точки зрения префиксного сравнения.

7. Все суффиксы между найденными границами начинаются с данного образца. Их позиции собираются в массив, переводятся к нумерации с единицы, сортируются по возрастанию и выводятся.

Сложность алгоритма

Пусть n — длина текста, m — длина текущего образца, k — количество вхождений текущего образца в текст.

Построение суффиксного массива выполняется за:

$$O(n \log n)$$

Память на суффиксный массив, классы эквивалентности и вспомогательные массивы:

$$O(n)$$

Для поиска одного образца выполняются два бинарных поиска по суффиксному массиву. Каждое сравнение суффикса с образцом в текущей реализации может занимать до $O(m)$, поэтому поиск границ выполняется за:

$$O(m \log n)$$

После этого найденные позиции копируются и сортируются:

$$O(k \log k)$$

Итоговая сложность поиска одного образца в текущей реализации:

$$O(m \log n + k \log k)$$

Если не учитывать сортировку позиций, то сама процедура поиска диапазона в суффиксном массиве занимает $O(m \log n)$.

Описание программы

Класс `TSuffixArray` — основной класс программы. Он хранит исходный текст в поле `text` и построенный суффиксный массив в поле `suffixArray`.

Метод `BuildSuffixArray` — строит суффиксный массив. Сначала к тексту добавляется символ `'\0'`, затем выполняется сортировка циклических сдвигов. На первом этапе используется сортировка подсчётом по символам. Далее на каждом шаге длина сравниваемых блоков удваивается, а сортировка выполняется по парам классов эквивалентности.

Массив `suffixOrder` — хранит порядок позиций после сортировки циклических сдвигов. После завершения алгоритма из него формируется итоговый `suffixArray`.

Массив `equivalenceClass` — хранит номер класса эквивалентности для каждой позиции строки. Две позиции имеют одинаковый класс, если соответствующие подстроки текущей длины равны.

Массивы `shiftedSuffixOrder` **и** `newEquivalenceClass` — вспомогательные массивы для очередного шага удвоения. `shiftedSuffixOrder` хранит новые позиции перед сортировкой подсчётом, а `newEquivalenceClass` — новые классы эквивалентности.

Метод `CompareSuffixWithPattern` — сравнивает суффикс текста, начинающийся с позиции `suffixPosition`, с образцом. Если суффикс начинается с образца, функция возвращает 0. Если суффикс лексикографически меньше образца, возвращается -1, иначе возвращается 1.

Метод `LowerBound` — бинарным поиском находит первый суффикс, который не меньше образца.

Метод `UpperBound` — бинарным поиском находит первый суффикс, который больше образца. Вместе методы `LowerBound` и `UpperBound` задают диапазон всех суффиксов, начинающихся с искомого образца.

Метод `SearchPattern` — выполняет поиск одного образца. Если образец пустой, возвращается пустой массив. Иначе находятся границы диапазона, из суффиксного массива извлекаются позиции вхождений, позиции переводятся к нумерации с единицы и сортируются по возрастанию.

Функция `main` — считывает первую строку как текст, строит объект `TSuffixArray`, затем построчно читает образцы до конца файла. Для каждого найденного образца выводится его номер и список позиций в формате, требуемом условием задачи.

Дневник отладки

1. Сначала был реализован общий каркас программы: считывание первой строки как текста, дальнейшее чтение образцов до конца файла и вывод номера образца только в

случае успешного поиска.

2. Затем был реализован алгоритм построения суффиксного массива через сортировку циклических сдвигов. Важным моментом стало добавление фиктивного символа `'\0'` в конец строки. Без него циклические сдвиги не превращаются напрямую в обычные суффиксы.

3. При реализации первого шага сортировки пришлось аккуратно приводить символы к `unsigned char`. Это нужно, чтобы символы с кодами больше 127 не превращались в отрицательные значения на системах, где `char` является знаковым.

4. На этапе удвоения были реализованы массивы `shiftedSuffixOrder` и `newEquivalenceClass`. Основная ошибка, которая обычно возникает в этом алгоритме, связана с переходом позиции через начало строки. Поэтому для элемента `shiftedSuffixOrder` добавляется длина строки, если после вычитания `substringLength` позиция стала отрицательной.

5. После построения массива `p` была удалена позиция фиктивного нулевого символа. Это важно, потому что выводить позицию, соответствующую добавленному служебному символу, нельзя.

6. Затем был реализован поиск одного образца. Для этого понадобилось написать функцию сравнения суффикса с образцом. Тонкое место здесь в том, что если образец полностью совпал с началом суффикса, функция должна вернуть 0, даже если сам суффикс длиннее образца.

7. Были реализованы два бинарных поиска: `LowerBound` и `UpperBound`. Они позволяют найти весь диапазон суффиксов, начинающихся с образца, а не только одно произвольное вхождение.

8. На примере из условия программа вывела:

```
1: 1

2: 2

3: 2, 6
```

   что совпало с ожидаемым результатом.

9. Дополнительно был проверен случай перекрывающихся вхождений. Например, для текста `aaaaa` образец `aaa` находится в позициях 1, 2 и 3. Это подтвердило, что программа не пропускает пересекающиеся совпадения.

Тест производительности

Для оценки эффективности решения были рассмотрены основные этапы работы программы: построение суффиксного массива и поиск образцов. Так как текст заранее известен, дорогостоящий этап построения выполняется один раз, после чего каждый новый образец ищется уже по готовому суффиксному массиву.

Асимптотические оценки приведены в таблице.

Этап	Время	Память
Построение суффиксного массива	$O(n \log n)$	$O(n)$
Сравнение суффикса с образцом	$O(m)$	$O(1)$
Поиск диапазона для одного образца	$O(m \log n)$	$O(1)$
Сортировка найденных позиций	$O(k \log k)$	$O(k)$
Полный поиск одного образца	$O(m \log n + k \log k)$	$O(k)$

Отдельно были проверены функциональные сценарии:

- образец встречается один раз;
- образец встречается несколько раз;
- образец не встречается в тексте и не выводится;
- вхождения образца пересекаются;
- образец совпадает со всем текстом;
- пустой образец не выводится;
- пустой текст не приводит к ошибке выполнения.

Пример из условия:

Вход	Выход

`abcdabc`
`abcd`
`bcd`
`bc`
	`1: 1`
	`2: 2`
	`3: 2, 6`

Для теста с перекрывающимися вхождениями:

Вход	Выход

`aaaaa`
`a`
`aa`
`aaa`
`b`
	`1: 1, 2, 3, 4, 5`
	`2: 1, 2, 3, 4`
	`3: 1, 2, 3`

Эти проверки подтверждают, что суффиксный массив корректно находит все позиции образцов, включая пересекающиеся вхождения.

Недочёты

- В текущей реализации сравнение образца с суффиксом выполняется напрямую за $O(m)$. Из-за этого поиск одного образца оценивается как $O(m \log n)$. Можно было бы использовать LCP и дополнительные структуры, чтобы уменьшить число повторных сравнений символов.
- После нахождения диапазона подходящих суффиксов позиции дополнительно сортируются. Это необходимо, потому что суффиксный массив упорядочен лексикографически, а не по возрастанию позиций в тексте. Такая сортировка добавляет $O(k \log k)$.

- Для индексов суффиксного массива используется тип `int`. Для обычных ограничений лабораторной работы этого достаточно, но для строк длиной больше $2^{31}-1$ потребовался бы тип большего размера.
- Алгоритм построения суффиксного массива работает с байтами строки. Это нормально для входа в виде обычных ASCII-строк, но для полноценной обработки UTF-8 нужно было бы заранее договориться, что считается одним символом.
- Пустой образец в программе не выводится. Формально это разумно, потому что у пустой строки было бы слишком много позиций вхождения, но в условии отдельно такой случай не разбирается.

Выводы

В результате выполнения лабораторной работы была реализована программа поиска образцов в заранее известном тексте с использованием суффиксного массива. Текст считывается один раз, после чего для него строится суффиксный массив за $O(n \log n)$. Далее каждый образец из входного файла ищется по готовому массиву с помощью двух бинарных поисков.

В ходе работы были реализованы построение суффиксного массива методом удвоения, сравнение суффикса с образцом, поиск левой и правой границы диапазона подходящих суффиксов, а также вывод найденных позиций в порядке возрастания.

Проверка на примере из условия подтвердила корректность формата вывода. Дополнительные тесты с пересекающимися вхождениями показали, что программа находит все появления образца в тексте, а не только непересекающиеся совпадения.

Таким образом, задача решена в полном объёме: построен суффиксный массив, реализован поиск всех входных образцов и обеспечен вывод позиций в требуемом формате.